

# AI Prompt Engineering & Development Report

A comprehensive repository and analysis of prompt engineering architectures utilized for autonomous classification, FastMCP context modeling, and system integration. Reviewing AI-assisted code generation, validation patterns, and human-in-the-loop review processes.

PREPARED BY:

AI Development Team

INTEGRATION:

FastMCP / Gemini API

DATE:

June 10, 2026

DEVELOPMENT STATE:

Verified and Deployed

STATUS:

COMPLETED

REPORT VERSION:

v1.0.0

## Introduction

This document serves as a complete repository of the prompts used during the planning, implementation, testing, debugging, and documentation of the Helpdesk Ticket Management MCP System.

### Why Prompts Are Documented

- **Reproducibility:** Enables full reproducibility of AI-assisted outputs and architectural scaffolding.
- **Knowledge Base:** Serves as an engineering design log for onboarding future developers.
- **Transparency:** Provides transparency for project reviews, evaluations, and compliance checks.

## AI-Assisted Development Flow

Development followed a hybrid approach where AI was used for scaffolding, boilerplate generation, documentation, testing assistance, and implementation guidance. Human developers performed integration, debugging, validation, security reviews, and final testing.

## Role of Human Review and Validation

All AI-generated code was manually reviewed. Developers verified database transactions, fixed path issues, added exception handling, validated security configurations, and ensured the system met business requirements.

## Prompt Registry - Architecture & Server Scaffolding

### PROMPT SPECIFICATION: HELPDESK AGENT SYSTEM CONCEPT AND ARCHITECTURE PLANNING

|                        |                                                                                                                                                                                                                                                              |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Purpose:               | Define the overall architecture of the helpdesk ticket management system.                                                                                                                                                                                    |
| Representative Prompt: | Design an enterprise-grade Help Desk Ticket Management System. The system must combine an autonomous AI Agent layer with a Model Context Protocol (MCP) server for tool management and a Google Gemini API connection for classifying incoming user tickets. |
| Output Produced:       | Architecture diagrams, workflow definitions, and component descriptions.                                                                                                                                                                                     |
| Files Impacted:        | README.md                                                                                                                                                                                                                                                    |
| Lessons Learned:       | Separating database logic from AI agent logic improves maintainability.                                                                                                                                                                                      |

### PROMPT SPECIFICATION: FASTMCP TICKET SERVER IMPLEMENTATION

|                        |                                                                                                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Purpose:               | Create a stdio-based MCP server exposing ticket management tools.                                                                                       |
| Representative Prompt: | Write a Python script server.py using FastMCP. Define tools to list tickets, create tickets, update tickets, add comments, and search a knowledge base. |
| Output Produced:       | MCP Server implementation.                                                                                                                              |
| Files Impacted:        | mcp_server/server.py                                                                                                                                    |
| Lessons Learned:       | Tool descriptions must be explicit because LLMs rely on them when selecting tools.                                                                      |

## Prompt Registry - Gemini API & Most Effective Prompts

| PROMPT SPECIFICATION: GEMINI STRUCTURED JSON CLASSIFICATION |                                                                                           |
|-------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Purpose:                                                    | Classify incoming issues and return structured data.                                      |
| Representative Prompt:                                      | Create a Gemini classifier that returns JSON with priority, category, and summary fields. |
| Output Produced:                                            | Gemini classification service.                                                            |
| Files Impacted:                                             | services/gemini_classifier.py                                                             |
| Lessons Learned:                                            | Forcing JSON output significantly reduces parsing errors.                                 |

### Most Effective Prompts Overview

|                                                                                                                |                                                                                                             |
|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>1. Gemini Structured JSON Classifier</b><br>Generated reliable machine-readable outputs for classification. | <b>2. ReAct Agent Workflow</b><br>Enabled multi-step reasoning, tool orchestration, and decision paths.     |
| <b>3. MCP Server Generator</b><br>Accelerated tool registration and database integration.                      | <b>4. Duplicate Detection Prompt</b><br>Improved issue matching accuracy via Jaccard similarity thresholds. |

### AI Assistance Summary Matrix

| AI-Assisted Tasks                                                                                                                                                                             | Human Developer Tasks                                                                                                                                                                                                   | Human Verification                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>MCP server scaffolding</li><li>Agent workflow design</li><li>Gemini integration</li><li>Documentation generation</li><li>Test case generation</li></ul> | <ul style="list-style-type: none"><li>Debugging &amp; path correction</li><li>Configuration management</li><li>Security &amp; RBAC validation</li><li>Database transaction checks</li><li>Performance testing</li></ul> | <ul style="list-style-type: none"><li>MCP tool testing</li><li>Database integrity verification</li><li>Gemini response checking</li><li>End-to-end workflow testing</li></ul> |

### Conclusion

AI significantly accelerated development by generating code, documentation, and architectural suggestions. Human oversight remained essential for debugging, validation, testing, configuration management, and ensuring production readiness. The hybrid cooperation proved highly effective in bringing the MCP Web Application to a robust state.